ARTIFICIAL INTELLIGENCE

TOPIC 4

Convolutional neural networks and transfer learning

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Calculate convolution sizes, parameter counts and receptive fields; explain pooling and common architecture modules; choose between feature extraction and fine-tuning; interpret training curves; distinguish classification, detection, segmentation and field regression.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Why images benefit from a structured network **4**

2 The convolution operation **4**

2.1 One dimension first 4

2.2 Two dimensions with channels 4

3 Stride, padding, dilation and output size **5**

3.1 What each setting changes 5

3.2 Worked size examples 5

4 Parameter counts and activation memory **5**

4.1 Standard convolution 6

4.2 Fully connected comparison 6

4.3 Memory is not only weights 6

5 Receptive fields grow across layers **6**

5.1 Worked stack 6

6 Downsampling, upsampling and 1-by-1 filters **7**

6.1 Pooling and strided convolution 7

6.2 Upsampling 7

6.3 Pointwise convolution mixes channels 7

7 Common architectures: learn the design idea **8**

7.1 Why small filters can be efficient 8

7.2 Depthwise-separable convolution 8

8 Residual connections: learn a correction **9**

8.1 Basic blocks, bottlenecks and matching shapes 9

9 Batch normalization: computation and parameter count **10**

9.1 The slide's convolution-plus-BN count 10

9.2 Why it can help 10

10 Training, inference and normalization alternatives **10**

10.1 Answers to the slide's quick check 11

11 Transfer learning: reuse a useful representation **11**

11.1 Frozen feature extraction . . . 12

11.2 Fine-tuning . . . 12

11.3 A worked transfer-learning setup . . . 12

12 Training checks and learning curves **12**

12.1 Initial sanity checks . . . 12

12.2 Interpreting the plots . . . 13

13 CNNs beyond whole-image classification **13**

13.1 Task determines output structure . . . 13

13.2 Detection: boxes and categories . . . 14

13.3 Pose regression . . . 14

13.4 Semantic and instance segmentation . . . 14

13.5 Video, multiple modalities and engineering data . . . 14

14 Worked slide exercise: a 240-by-240 input **14**

14.1 Case A: valid convolutions and 2-by-2 pooling . . . 15

14.2 Case B: same-size convolutions . . . 15

15 Revision questions and answers **15**

15.1 Questions . . . 15

15.2 Answers . . . 15

16 Source guide and study connection to Lab 3 **16**

1 Why images benefit from a structured network

Teacher slides: 4.1. CNNs, pp. 3–11.

A color image is an array with height, width and channels. A 32×32 RGB image has 3072 scalar entries. Flattening it into a vector for a fully connected layer loses the explicit grid structure and gives a separate parameter for every input-to-output connection.

Nearby pixels often jointly describe an edge or texture, and the same pattern can occur in different locations. Convolutional networks encode these assumptions using **local connectivity** and **shared weights**. A small filter searches for a pattern across the grid, producing a feature map. Later layers combine patterns over larger receptive fields.

These are inductive biases rather than universal facts about all data. A convolution usually responds to a translated pattern with a translated feature map: this is translation *equivariance*. It does not automatically make the whole network invariant to position. Striding, image boundaries and a fully connected head can change how that behavior works.

Track four different quantities

Spatial size describes the grid; channel count describes features at each location; trainable parameters describe learned coefficients; receptive field describes which input locations can affect one output. Changing one need not change the others in the same way.

2 The convolution operation

Teacher slides: 4.1. CNNs, pp. 12–19, 24–30.

2.1 One dimension first

Slide a filter over neighboring input values and take a weighted sum. For an input $(1, 2, 3, 4, 5)$ and three weights $(1, 0, -1)$, stride one and no padding, the outputs are

$$(1 - 3,\ 2 - 4,\ 3 - 5) = (-2, -2, -2).$$

The same three weights are used at all positions. A bias can be added, followed by an activation such as ReLU. Strict mathematical convolution reverses the filter; common neural-network layers perform cross-correlation instead. For learned filters, this naming distinction does not prevent learning the desired patterns.

2.2 Two dimensions with channels

For output channel o and output location (i, j),

$$z_{i,j,o} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{u,v,c,o}\, x_{iS_h+uD_h-P_h,\, jS_w+vD_w-P_w,\, c}.$$

Interpret input locations outside the valid grid using the stated padding rule. A standard output filter sums across *all* input channels. For an RGB input, one 3×3 output filter contains $3 \times 3 \times 3$ weights, not merely nine. Different output filters learn different features.

The preactivation z and activated feature map $a(z)$ have the same shape. Spatial width and height tell where features occur; the output channel index tells which filter produced them.

3 Stride, padding, dilation and output size

Teacher slides: 4.1. CNNs, pp. 15–18.

3.1 What each setting changes

Kernel size determines how many learned positions a filter has. **Stride** determines how far the filter moves between output locations. **Padding** extends the grid at the boundary. **Dilation** spaces filter positions apart, covering a larger area without adding learned weights.

For one spatial dimension, let input size be W, kernel size K, stride S, dilation D, and symmetric padding P on each side. Effective kernel width is

$$K_{\text{eff}} = D(K-1)+1.$$

Then

$$\boxed{W_{\text{out}} = \left\lfloor \frac{W + 2P - K_{\text{eff}}}{S} \right\rfloor + 1}.$$

Apply this separately to height and width. A valid output requires an effective kernel that fits the padded input. If padding is asymmetric, replace $2P$ by the sum of the two side paddings.

Using the slide size formula

Use integer floor, as displayed on p. 17, when the division is not exact. Dilation $D = 1$ means no gaps. Same padding preserves size with stride one under the usual conventions; for larger strides, output is typically a rounded-downsampling size, not unchanged input size.

3.2 Worked size examples

Input and settings	Calculation	Output
$W = 8, K = 3, S = 2, P = 1, D = 1$	$\lfloor (8 + 2 - 3)/2 \rfloor + 1$	4
$W = 8, K = 5, S = 1, P = 2, D = 1$	$(8 + 4 - 5) + 1$	8
$W = 8, K = 3, S = 1, P = 0, D = 2$	$K_{\text{eff}} = 5$; $(8 - 5) + 1$	4
$W = 32, K = 3, S = 1, P = 1, D = 1$	$(32 + 2 - 3) + 1$	32

The first two reproduce the arithmetic of the slide examples with the floor made explicit. Dilation changes the effective footprint, not the number of learned filter positions.

4 Parameter counts and activation memory

Teacher slides: 4.1. CNNs, pp. 29–30, 48–50, 106.

4.1 Standard convolution

For C_{in} input channels and C_{out} output channels, with one bias per output channel,

$$N_{\text{params}} = C_{\text{out}}(K_h K_w C_{\text{in}} + 1).$$

Omit the $+1$ if there is no bias. Stride, padding, dilation and spatial image size do not directly change this count. They change activation sizes and computation.

For 3×3 filters taking RGB to 48 channels, the count is $48(9 \cdot 3 + 1) = 1344$. A second 3×3 layer taking 48 channels to 48 has $48(9 \cdot 48 + 1) = 20784$. Identical spatial filter size does not imply identical parameter count.

4.2 Fully connected comparison

A dense mapping from a flattened $32 \times 32 \times 3$ image to 64 units has $64(3072 + 1) = 196672$ parameters. A convolution from 3 to 64 channels with 3×3 filters has $64(27 + 1) = 1792$. The outputs differ: the convolution creates a spatial feature map rather than only a 64-entry vector.

4.3 Memory is not only weights

For batch size B, an activation tensor contains $BHWC$ scalar values. With float32 storage, its raw footprint is $4BHWC$ bytes. Training also stores intermediate activations, gradients and optimizer state. For $B = 16$, $H = W = 120$, $C = 48$, just this tensor contains 11,059,200 values, about 42.2 MiB.

Downsampling can reduce activation memory even while later layers increase channels. This explains why the slides ask about memory and batch size separately from parameter counts. A small parameter count does not imply that a high-resolution model is cheap to train.

5 Receptive fields grow across layers

Teacher slides: 4.1. CNNs, pp. 20–23.

The receptive field of an output is the part of the original input that can affect it. A second local layer sees combinations of first-layer receptive fields, so its effective input footprint grows.

Track r_ℓ, receptive-field width, and j_ℓ, spacing between adjacent outputs measured in original input pixels. Start with $r_0 = 1$, $j_0 = 1$. For effective filter size $K_{\text{eff},\ell}$ and stride S_ℓ,

$$r_\ell = r_{\ell-1} + (K_{\text{eff},\ell} - 1)j_{\ell-1}, \qquad j_\ell = j_{\ell-1} S_\ell.$$

This calculation explains the slide diagrams for a simple sequential stack; branching architectures require tracing each path.

5.1 Worked stack

Take 3×3 convolution with stride 1; 2×2 pooling with stride 2; then 3×3 convolution with stride 1:

Stage	r	j
Input	1	1
Conv 3, stride 1	3	1
Pool 2, stride 2	4	2
Conv 3, stride 1	8	2

The final nominal receptive field is 8×8, and neighboring outputs are centered two original pixels apart. With padded boundaries, part of that nominal field may consist of padding rather than observed pixels. The learned influence inside the theoretical footprint need not be uniform.

6 Downsampling, upsampling and 1-by-1 filters

Teacher slides: 4.1. CNNs, pp. 31–36.

6.1 Pooling and strided convolution

Max pooling takes the largest value in each local region; average pooling takes their mean. Ordinary pooling acts separately by channel and has no trainable weights. Strided convolution instead learns a local weighted reduction while downsampling.

For the patch $\begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix}$, max pooling gives 4 and average pooling gives 2.5. Pooling reduces spatial detail; it can improve robustness to small shifts but does not guarantee complete translation invariance.

6.2 Upsampling

The slides show duplication, interpolation, max-unpooling and transposed convolution. Duplication repeats values; interpolation estimates values between existing grid points; max-unpooling can place values according to stored pooling positions. Transposed convolution uses a learnable operation related to the transpose of the convolution's linear map. It is not generally an inverse that recovers information lost by downsampling.

Upsampling is needed when the output must return to a fine spatial grid, as in segmentation. It enlarges the representation but cannot recover discarded detail without information learned or retained elsewhere.

6.3 Pointwise convolution mixes channels

A 1×1 convolution takes the channel vector at each location and applies the same affine mapping at every location. Its count is $C_{\text{out}}(C_{\text{in}}+1)$ with biases. It can reduce or increase channel dimension without directly mixing neighboring spatial pixels.

For 48 input and 16 output channels, a pointwise layer has $16(48+1) = 784$ parameters. It can serve as a bottleneck before a more expensive spatial convolution.

7 Common architectures: learn the design idea

Teacher slides: 4.1. CNNs, pp. 37–66.

Architecture or module	Main idea emphasized in the slides
AlexNet	A layered convolution/pooling classifier illustrating the large-scale image-recognition breakthrough.
DeConvNet visualization	Inspect feature-map activations and map selected activity back toward image space to understand learned responses.
VGG	Deeper stacks using small spatial filters. Several small filters build a large receptive field with intermediate nonlinearities.
Inception / GoogLeNet	Parallel branches with different operations, combined into a feature representation; bottlenecks can reduce cost.
ResNet	Residual connection $y = x + F(x)$, providing a direct path for information and gradients.
DenseNet	Concatenate earlier feature maps into later layers, encouraging reuse rather than adding only one residual branch.
MobileNet	Depthwise spatial filtering followed by pointwise channel mixing.
EfficientNet	Scale depth, width and input resolution together in a coordinated way.
ViT and ConvNeXt	The slides contrast patch-based attention with a modernized convolutional design; they do not supply a transformer derivation.

Do not memorize these as interchangeable deeper versions of the same model. Their connectivity and computation differ. For a residual sum, tensor shapes must agree or a projection must align them. Concatenation instead increases channel count. Both alter gradient paths, but neither guarantees successful training on every task.

7.1 Why small filters can be efficient

Two stride-one 3×3 layers have a 5×5 receptive field. If input, intermediate and output widths are all C, their kernel weights total $18C^2$, compared with $25C^2$ for one 5×5 layer. They also include an extra activation. This comparison assumes equal channel widths and ignores biases; other widths can change the trade-off.

7.2 Depthwise-separable convolution

For a depthwise layer with one filter per input channel, followed by pointwise mixing to C_{out} channels, the weight count is

$$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}},$$

instead of $K^2 C_{\text{in}} C_{\text{out}}$ for standard convolution. For $K = 3, C_{\text{in}} = 32, C_{\text{out}} = 64$, this is $288 + 2048 = 2336$ rather than 18432 weights. Bias choices add separate terms. Lower arithmetic cost does not by itself guarantee proportional wall-clock speedup on every device.

8 Residual connections: learn a correction

Teacher slides: 4.1. CNNs, pp. 51–54, 60–62.

Increasing a plain network's depth can make optimization harder. If a deeper model has *higher training error*, this is not by itself the usual evidence for overfitting: overfitting concerns poor generalization despite fitting training data well.

A residual block learns $F(x)$ and adds a shortcut:

$$y = x + F(x).$$

If preserving the input is appropriate, the residual branch can learn $F(x) = 0$. Before any activation applied after the sum, the block then represents identity. This makes identity-like solutions easier to represent than requiring several ordinary layers to learn them indirectly.

For the residual sum, the Jacobian is

$$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}.$$

The identity term supplies a direct gradient contribution. It helps optimization but does not prove that gradients can never vanish, explode or cancel. Additional activations and projections must be included when differentiating a full block.

An in-principle argument is not a training guarantee

If the shapes and activation arrangement permit identity, a residual extension can represent the previous mapping by choosing a zero residual. This does not guarantee that training finds that solution or that the deeper model generalizes better.

8.1 Basic blocks, bottlenecks and matching shapes

A basic ResNet block uses two spatial convolutions. A bottleneck uses a 1×1 projection to reduce channels, a spatial convolution at that reduced width, then a 1×1 projection to expand again. The expensive spatial operation acts on fewer channels.

The slide compares two 3×3 convolutions at width 256 with a bottleneck $256 \to 64 \to 64 \to 256$, ignoring biases and normalization:

$$2(9)(256^2) = 1{,}179{,}648, \qquad 256(64) + 9(64^2) + 64(256) = 69{,}632.$$

The bottleneck uses about 17 times fewer convolution weights under these assumptions, not necessarily 17 times less execution time.

Residual addition requires matching spatial and channel dimensions. If a stage changes resolution or channels, a projection shortcut, often a strided 1×1 convolution, can match the shape. An identity shortcut has no learned parameters; a projection shortcut does.

Adding tensors preserves their common shape. Concatenating along channels adds their channel counts. Two $16 \times 16 \times 32$ feature maps give $16 \times 16 \times 32$ by addition, or $16 \times 16 \times 64$ by channel concatenation. DenseNet and U-Net use concatenation in important parts of their designs; it is not interchangeable with residual addition.

9 Batch normalization: computation and parameter count

Teacher slides: 4.1. CNNs, pp. 55–56, 58.

Batch normalization (BN) controls activation scale using statistics inside the network, then applies a learned affine transformation. For values $a_1, \ldots, a_m$ in a normalization group,

$$\mu_B = \frac{1}{m}\sum_i a_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_i (a_i - \mu_B)^2, \qquad y_i = \gamma \frac{a_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

In a CNN, ordinary BN computes statistics separately for each channel over batch and spatial positions. For shape $N \times C \times H \times W$, one channel's group contains NHW values.

For C channels, C learned scales γ and C learned offsets β give $2C$ trainable parameters. Running means and variances are stored statistics, not coefficients learned by gradient descent. Implementations may also store bookkeeping such as a batch counter.

For values $(2, 4)$, the batch mean is 3 and population variance is 1. Ignoring ϵ for this arithmetic, normalization gives $(-1, 1)$. If $\gamma = 2, \beta = 0.5$, outputs are $(-1.5, 2.5)$. The learned affine step means final outputs do not have to retain mean zero and variance one.

9.1 The slide's convolution-plus-BN count

For a 3×3 convolution from 64 to 128 channels followed directly by BN, use no convolution bias:

$$N_{\text{conv}} = 9(64)(128) = 73{,}728, \qquad N_{\text{BN,trainable}} = 2(128) = 256.$$

Total trainable parameters are 73,984. Running statistics are not part of the trainable count.

A convolution bias immediately before BN is usually redundant: batch centering removes its constant channel shift and BN supplies a learned offset. If an exercise explicitly retains that bias, count the additional 128 stored trainable values rather than silently changing the architecture.

9.2 Why it can help

BN often makes optimization less sensitive to activation scale and can permit larger learning rates. Batch statistics also introduce noise with a mild regularizing effect. Initialization only sets the starting scale; normalization continues as weights change. The historical explanation in terms of internal covariate shift is not the only explanation of its benefits, as the slides themselves note.

10 Training, inference and normalization alternatives

Teacher slides: 4.1. CNNs, pp. 57–59, 62.

During ordinary BN training, normalization uses current mini-batch statistics and updates running estimates. During ordinary inference with tracked statistics, it uses stored estimates. Predictions then do not depend on which other samples share the evaluation batch, apart from other numerical or model effects.

For a convention in which m weights the new observation,

$$\mu_{\text{run,new}} = (1 - m)\mu_{\text{run,old}} + m\mu_B.$$

If the old mean is 2, the batch mean is 4 and $m = 0.1$, the new mean is 2.2. Some conventions name the complementary coefficient, so inspect the equation.

Evaluation mode and disabling gradient tracking are different controls. Turning off gradients alone does not switch BN to stored statistics or deactivate dropout. Freezing backbone weights also does not necessarily freeze BN running estimates if the backbone remains in training mode.

At inference, BN with fixed statistics is affine per channel and can often be folded into a preceding convolution. Small or unrepresentative training batches can give noisy statistics. The batch-size ranges on the slides are practical heuristics, not universal boundaries.

Method	Values sharing mean/variance	Course connection
Batch norm	Per channel, across batch and spatial positions in a CNN.	Training versus running-statistic inference.
Layer norm	Within a sample's chosen feature axes; usually each token's feature coordinates in a transformer.	Transformers and recurrent models.
Instance norm	Spatial positions within one sample and channel.	Image stylization.
Group norm	Spatial positions and a group of channels within one sample.	Small-batch CNN tasks.

The methods differ chiefly in which axes supply statistics. Transformer layer norm does not need a running batch mean and normally applies the same per-token computation during training and inference.

10.1 Answers to the slide's quick check

1. Higher training error in the deeper model points to optimization/representation issues; it is not alone evidence of overfitting.
2. A compatible residual block can represent identity through a zero residual, but successful training is not guaranteed.
3. The bias-free 3×3, $64 \to 128$ convolution plus BN has 73,984 trainable parameters.
4. Unexpected evaluation-batch dependence suggests checking training/evaluation mode and BN statistics; it is a diagnostic clue, not the only possible cause.
5. Layer norm uses within-sample feature statistics rather than BN's cross-sample per-channel statistics, and does not ordinarily rely on running batch estimates at inference.

11 Transfer learning: reuse a useful representation

Teacher slides: 4.1. CNNs, pp. 67–74, 102.

11.1 Frozen feature extraction

Take a pretrained network, remove or bypass its original task-specific output, and use a feature vector to train a new classifier or regressor. Keep the feature extractor's weights fixed. The slides illustrate a 4096-dimensional AlexNet feature vector used by a later learning algorithm.

This can be attractive with few labeled target examples or limited compute. It still requires matching input preprocessing and choosing a representation appropriate to the new task. Pretraining does not make every feature suitable for every domain.

11.2 Fine-tuning

Replace the output head for the new classes and update some or all pretrained layers on target data. A practical strategy is to fit the new head first, then gradually unfreeze useful layers with a suitably small learning rate. This refines the representation rather than only learning a new readout.

Situation	Reasonable starting point
Few examples, similar domain	Freeze much of the representation and fit a small head.
More data, similar domain	Fine-tune later blocks, then validate whether broader adaptation helps.
Different domain	Investigate which features transfer; adapt more layers if data supports it.
Large suitable dataset and sufficient resources	Compare fine-tuning with training from scratch.

These are starting points, not guarantees. Use the target validation split to choose frozen layers, learning rates and training duration. Inference-mode behavior also matters: dropout and batch-normalization statistics can change when a backbone is run in training mode.

11.3 A worked transfer-learning setup

Suppose a pretrained extractor returns $d = 512$ features and the target has $C = 5$ classes. A new affine head has $5(512 + 1) = 2565$ trainable parameters. Freezing the backbone leaves that count trainable, but its stored weights and forward computation remain necessary. Unfreezing a block increases the trainable count and requires backward activation storage.

12 Training checks and learning curves

Teacher slides: 4.1. CNNs, pp. 75–81.

12.1 Initial sanity checks

For C equally likely classes with near-uniform initial predictions, mean cross-entropy should be around $\log C$. This is a reference point, not a requirement for an imbalanced or nonuniform initial setup. Check that increasing a nonnegative penalty increases the reported total objective when predictions and parameters are fixed.

Try to overfit a tiny training subset with weak regularization. Failure can reveal an unsuitable optimizer configuration, wrong labels, shape errors, disconnected gradients or inadequate capacity. Success shows basic ability to fit those samples; it does not demonstrate good validation performance.

12.2 Interpreting the plots

Observation	What to investigate
Training loss stays flat	Learning rate, labels, gradient flow, frozen weights or loss choice.
Loss explodes or is nonfinite	Excessive step size, input scale, unstable calculations or problematic samples.
Training improves, validation worsens	Overfitting, distribution mismatch, leakage in the experiment or inappropriate augmentation.
Both training and validation remain poor	Insufficient capacity or optimization, wrong representation or a difficult task.
Noisy but decreasing training loss	Small batches may explain noise; compare epoch averages and validation trend.

Inspect first-layer filters and activations for additional clues, but an attractive filter image is not a performance metric. The slides recommend learning-rate decay and a coarse-to-fine hyperparameter search when resources allow. Choose configurations using validation results and reserve final test evaluation.

13 CNNs beyond whole-image classification

Teacher slides: 4.1. CNNs, pp. 82–105.

13.1 Task determines output structure

Task	Output	Slide examples
Classification	A class distribution for the image	ImageNet-style classifier.
Detection	Object labels and bounding boxes	R-CNN family; YOLO.
Pose regression	Continuous position and orientation	PoseNet.
Semantic segmentation	A category at every pixel	FCN; encoder–decoder / SegNet.
Instance segmentation	A separate object mask for each instance	Mask R-CNN.
Field regression	Continuous values on a grid	Simulation surrogate for flow fields.

The feature extractor can be similar while the head, target labels and loss change substantially. One classification label per image does not directly train a pixelwise segmentation model.

13.2 Detection: boxes and categories

Detection combines recognition and localization. Bounding-box coordinates are regression outputs; classes are classification outputs. Faster R-CNN learns region proposals through a proposal network; YOLO predicts detections in one main pass. The slide's 448×448 input and 7×7 grid refer to the historical YOLO example shown, not all models carrying that name.

Nonmaximum suppression removes redundant overlapping predictions according to a heuristic and a chosen score/overlap rule. It is postprocessing, not the same thing as the network learning a box. A detector can be correct about category and wrong about localization, which is why whole-image accuracy alone is insufficient.

13.3 Pose regression

PoseNet replaces a classification output with position and orientation estimates. The slides use three position coordinates and a four-component orientation quaternion: seven output numbers representing a six-degree-of-freedom pose. Quaternion representation imposes normalization and sign-equivalence considerations; seven numbers do not mean seven independent physical degrees of freedom.

13.4 Semantic and instance segmentation

Semantic segmentation assigns a class to each pixel. Two adjacent cars can belong to one semantic category without the output separating their identities. Instance segmentation distinguishes individual objects as well as categories.

An encoder compresses spatial resolution and builds features; a decoder restores a dense output grid. Upsampling and retained spatial information help produce fine predictions. FCNs and encoder–decoder networks replace a fixed global classification readout with spatial outputs. Mask R-CNN adds per-instance masks to object detection.

13.5 Video, multiple modalities and engineering data

The slides explore combining frames and combining appearance with pose or motion information. Simply aggregating frame features can discard ordering; sequential architectures are mentioned as later topics. The supplied deck does not yet derive RNNs, temporal convolutional networks or transformers.

Convolution also applies to one-dimensional sensor signals and three-dimensional volumes. Engineering examples include defect recognition, climate-event detection, grid-based flow-field prediction and mapping material microstructure to properties. For a simulation surrogate, train on simulated input–output pairs and predict a continuous field. A surrogate's speed is useful only alongside accuracy on relevant held-out geometries and conditions; it inherits limitations of its data coverage.

14 Worked slide exercise: a 240-by-240 input

Teacher slides: 4.1. CNNs, p. 106.

The teacher asks for the feature-map size after Conv1, pooling and Conv2, using a $240 \times 240 \times 3$ input and 48 filters of size 3×3 in each convolution. The slide does not fully specify padding or

the pooling window. State assumptions before giving a single size.

14.1 Case A: valid convolutions and 2-by-2 pooling

Assume both convolutions use stride 1 and no padding; pooling uses window 2, stride 2, no padding:

Stage	Spatial calculation	Tensor shape
Input	–	$240 \times 240 \times 3$
Conv1	$240 - 3 + 1 = 238$	$238 \times 238 \times 48$
Pool	$\lfloor (238 - 2)/2 \rfloor + 1 = 119$	$119 \times 119 \times 48$
Conv2	$119 - 3 + 1 = 117$	$117 \times 117 \times 48$

Parameter counts are 1344 for Conv1, zero for pooling and 20784 for Conv2, totaling 22128 with biases. The final receptive field is 8×8 under these assumptions.

14.2 Case B: same-size convolutions

If each 3×3 convolution uses padding 1, sizes become $240 \times 240 \times 48$, then $120 \times 120 \times 48$, then $120 \times 120 \times 48$. The parameters are unchanged: padding adds no learned coefficients.

For the question about the fifth convolution's parameters, its input and output channel counts must be known from the architecture. A layer's index alone is insufficient to calculate the number. Use the same formula with the actual channels, kernel size and bias setting.

15 Revision questions and answers

15.1 Questions

1. What assumptions distinguish convolution from a dense image layer?
2. For $W = 15, K = 3, S = 2, P = 1, D = 1$, what is output width?
3. How many parameters are in a biased 3×3 convolution from 8 to 16 channels?
4. Does doubling image width double convolutional parameter count?
5. What changes when dilation goes from 1 to 2 for a three-position kernel?
6. Do two stride-one 3×3 filters have only a 3×3 receptive field?
7. Does ordinary max pooling mix channels or learn weights?
8. What does a 1×1 convolution do?
9. What is the difference between feature extraction and fine-tuning?
10. Which output distinguishes two adjacent cars as separate objects?

15.2 Answers

1. Local connectivity and weights shared across spatial locations.

2. $\lfloor (15 + 2 - 3)/2 \rfloor + 1 = 8$.
3. $16(9 \cdot 8 + 1) = 1168$.
4. No; activation size and computation change instead.
5. Effective width changes from 3 to 5 while there are still three learned positions.
6. No; their nominal receptive field is 5×5.
7. Neither. It reduces each channel's spatial map independently.
8. Applies a shared affine channel mixing at every spatial location.
9. Feature extraction freezes the pretrained representation; fine-tuning updates some of it using target data.
10. Instance segmentation; semantic segmentation alone assigns only a category per pixel.

16 Source guide and study connection to Lab 3

`4.1. CNNs.pdf`: operations and geometry (pp. 3–36), architectures including expanded residual/normalization material (pp. 37–66), transfer learning (pp. 67–74), training diagnostics (pp. 75–81), task extensions and engineering applications (pp. 82–105), exercises and demonstrations (pp. 106–107), and the Lab 3 outline (pp. 108–110).

The lab outline emphasizes understanding a network, implementing a toy CNN and fine-tuning a pretrained architecture. The shape calculations, transfer-learning choices and training checks here support those objectives. Dataset and setup instructions in the slides are course-time instructions; these notes do not infer additional assignment requirements or current deadlines.